



nextwork.org

Deploy an App Across Accounts



Arpita Chowdhury

Hello from Ahmed at NextWork!

If you can see this, you've deployed my app... nice work!

You've unlocked my secret code: Avatar-The Last Airbender

Something I've learnt about you today is that you aspire to be a great Cloud Engineer!!

And here's a special image chosen by me: From my most recent movie- Avatar Fire & Ash





Introducing Today's Project!

In this project, I will demonstrate how to build a Docker container image for a custom application, store it securely in Amazon ECR, and share the container image so others can pull and launch the app. I'm doing this project to learn how container images are managed, stored, and shared in the cloud using AWS services, which reflects real-world workflows used by cloud and DevOps teams.

What is Amazon ECR?

Amazon ECR is a fully managed container image registry service in AWS, and it is useful because it provides a secure and scalable place to store, version, and share Docker container images. I used ECR to push my container image to the cloud so it could be pulled and deployed by Player A and other AWS services.

One thing I didn't expect...

One thing I didn't expect in this project was how many supporting artifacts ECR creates when you push a single Docker image, such as image indexes and metadata, which helped me understand how Docker handles multi-architecture images in real-world deployments.



Arpita Chowdhury

NextWork Student

nextwork.org

This project took me...

This project took me 5 hours, and my biggest learning was how Docker and Amazon ECR work together to store, share, and deploy container images.



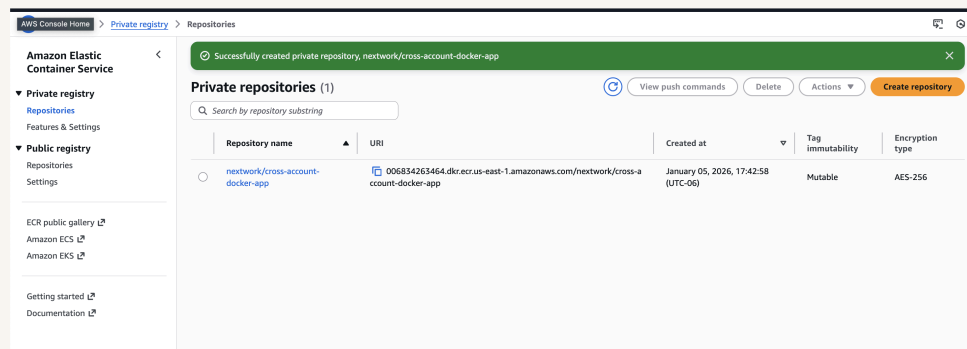
Creating a Docker Image

I set up a Dockerfile and an HTML file. One defines how the container is built, and the other defines what the web app displays.

My Dockerfile tells Docker to start from the Nginx base image and copy my custom index.html into the web server's default directory so the container serves my web page when it runs.

I also set up an ECR repository

ECR stands for Elastic Container Registry, and it's important because it lets teams store and share container images reliably in the cloud.





Set Up AWS CLI Access

AWS CLI can let me run ECR commands

AWS CLI is a tool for managing AWS through the terminal, and it asked for my credentials to securely authenticate and authorize my actions.

To enable CLI access, I used my already set up IAM user with ECR permissions and created an access key so the AWS CLI can securely authenticate and push images to Amazon ECR. I used my existing IAM user for this setup.

I used `aws configure` and provided my access key, secret key, region, and output format to authenticate the AWS CLI.

```
arpitachowdhury@Arpita-MacBook-Air DockerECR % aws --version
aws-cli/2.32.29 Python/3.13.11 Darwin/24.6.0 exe/arm64
arpitachowdhury@Arpita-MacBook-Air DockerECR % aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin 006834263464.dkr.ecr.us-east-1.amazonaws.com
An error occurred (UnrecognizedClientException) when calling the GetAuthorizationToken operation: The security token included in the request is invalid.
error: cannot perform an interactive login from a non TTY device
arpitachowdhury@Arpita-MacBook-Air DockerECR % aws configure
AWS Access Key ID [*****IMGP]:
```



Pushing My Image to ECR

Push commands are `docker tag` to label my image with the ECR repository address and `docker push` to upload the image to Amazon ECR.

There are three main push commands

I authenticated Docker with ECR using the `aws ecr get-login-password` command piped into `docker login`.

To push my container image, I ran the command `docker push 006834263464.dkr.ecr.us-east-1.amazonaws.com/nextwork/cross-account-docker-app:latest`. Pushing means uploading my local Docker image to Amazon ECR so it's stored in the cloud and can be pulled and deployed by others or AWS services.

When I built my image, I tagged it with the label `nextwork/cross-account-docker-app:latest`, which means the image is named and versioned locally and can be mapped to my ECR repository URL before pushing it to the cloud.



Resolving Permission Issues

When I tried pulling my project buddy's container image for the first time, I saw a 403 Forbidden error. This was because my AWS account did not yet have permission to access my buddy's Amazon ECR repository, since ECR repositories are private by default and require explicit cross-account permissions to allow image pulls.

We updated our ECR repository policies to allow cross-account image pulls by granting each other the required permissions.

```
arpitachowdhury@Arpita-MacBook-Air DockerECR % docker pull 007399981516.dkr.ecr.ap-northeast-1.amazonaws.com/nextwork/cross-account-docker-app:latest
Error response from daemon: unknown: failed to resolve reference "007399981516.dkr.ecr.ap-northeast-1.amazonaws.com/nextwork/cross-account-docker-app:latest": unexpected status from HEAD request to https://007399981516.dkr.ecr.ap-northeast-1.amazonaws.com/v2/nextwork/cross-account-docker-app/manifests/latest: 403 Forbidden
arpitachowdhury@Arpita-MacBook-Air DockerECR %
```



Deploying the App

I used Elastic Beanstalk to deploy and run my project buddy's containerized application directly from Amazon ECR without manually managing servers. When I set up the Elastic Beanstalk application, I configured settings such as the Docker platform, application code as a ZIP file containing Dockerrun.aws.json, the EC2 instance profile with ECR access, public IP enablement, single-instance environment type, deployment policy set to AllAtOnce, and disabled managed updates to keep the deployment simple.

While setting up for deployment, I created a new role for the Elastic Beanstalk service role and an EC2 instance profile. The role has the permission to provision and manage the required AWS resources (like EC2, security groups, logging, and deployments), and the EC2 instance profile has permission to pull container images from Amazon ECR. I also gave Elastic Beanstalk access to ECR by attaching ECR read/pull permissions so the EC2 instance can download and run my buddy's Docker image during deployment.

The Dockerrun.aws.json file is a configuration file that tells AWS Elastic Beanstalk how to deploy and run a Docker container. My file tells Elastic Beanstalk to pull Player A's container image from Amazon ECR (using the image URI), always use the latest version (Update: true), and run the container with port 80 exposed so the web app can be accessed through the Beanstalk environment.



```
{ } Dockerrun.aws.json ×
Users > arpitachowdhury > Desktop > DockerECR > { } Dockerrun.aws.json > ...
1
2 "AWSEBDockerrunVersion": "1",
3 "Image": {
4   "Name": "007399901516.dkr.ecr.ap-northeast-1.amazonaws.com/nextwork/cross-account-docker-app:latest",
5   "Update": "true"
6 },
7 "Ports": [
8   {
9     "ContainerPort": "80"
10  }
11 ]
12
```



Resolving Deployment Issues

There was an error when I visited my running application because the Docker container failed to deploy properly, so Elastic Beanstalk had no running service to respond to the request.

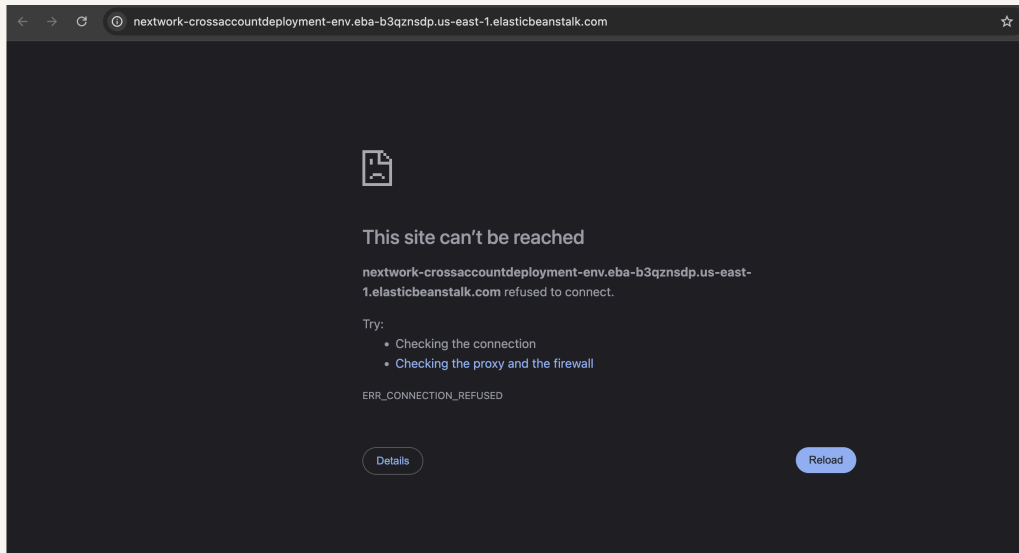
To fix the permissions error, my buddy and I updated our Amazon ECR repository policies to allow cross-account image pulls by adding each other's AWS account/role as an allowed principal and granting the required read actions (ecr:BatchCheckLayerAvailability, ecr:BatchGetImage, and ecr:GetDownloadUrlForLayer). My ECR repository is private by default, so we had to explicitly grant permissions so the other account (and the Elastic Beanstalk EC2 instance role) could authenticate and pull the container image successfully.



Arpita Chowdhury

NextWork Student

nextwork.org





nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

